

Week-8 (DAY-36) Hands-On – Keerthivasan R

Problem: Contact Management (Web API + EF Core Code First + JWT Authentication)

Scenario:

A growing organization wants to build a secure and scalable Contact Management System to manage employee contact details across multiple companies and departments.

Previously, the system was built using an in-memory collection. Now, the organization wants to upgrade it to a database-driven application using Entity Framework Core (Code First Approach).

Additionally, due to security requirements, the system must implement:

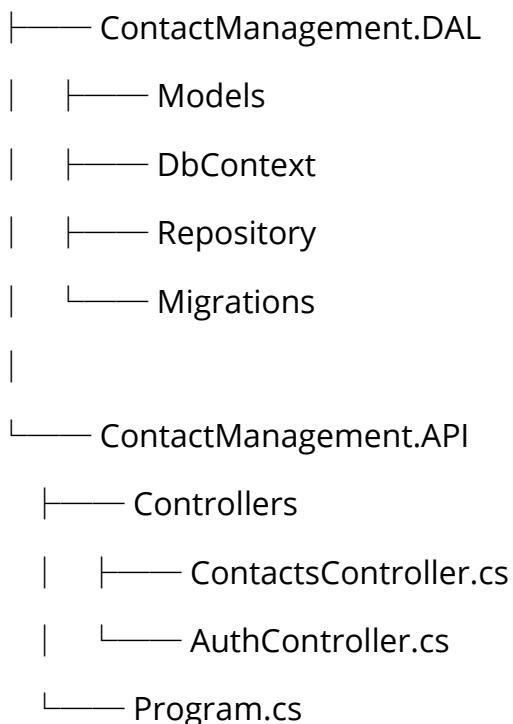
-  User Authentication (JWT Token Based)
-  Role-Based Authorization (Admin/User)

To maintain clean architecture and separation of concerns, the application must follow a 2-Layer Architecture:

- **DAL (Data Access Layer) → Handles database + repository logic**
- **API Service Layer → Exposes Web APIs + Authentication**

Solution Structure

Solution: ContactManagementSystem



Requirements:

1. Model Classes (DAL Layer)

◇ Model Class → ContactInfo (Main Entity)

Name	Type	Modifier
ContactId	int	public
FirstName	string	public
LastName	string	public
EmailId	string	public
MobileNo	long	public
Designation	string	public
CompanyId	int	public (FK)
DepartmentId	int	public (FK)

◇ Model 2 → Company

Name	Type
CompanyId	int
CompanyName	string

◇ Model 3 → Department

Name	Type
DepartmentId	int
DepartmentName	string

📍 Functional Requirements

🔑 Authentication Module

☑ Register User

- Create new user with Role (Admin/User)

☑ Login User

- Validate credentials
- Generate **JWT Token**

🔒 Authorization Rules (Role-Based Access)

Operation	Admin	User
Create Contact	☑	✗
Update Contact	☑	✗
Delete Contact	☑	✗
View Contacts	☑	☑

📍 Contact Management (CRUD)

☑ Create Contact

- Add contact to database (Admin only)

☑ Read Contact

- Get all contacts
- Get contact by ID
- Include **Company & Department details**

☑ Update Contact

- Update existing contact (Admin only)

☑ Delete Contact

- Delete contact (Admin only)

📍 Relationship Requirements

- One **Company** → **Many Contacts**
- One **Department** → **Many Contacts**

🔗 Students must implement:

- Foreign Keys

- Navigation Properties

Technical Constraints

Architecture (MANDATORY)

◊ Layer 1: DAL (Class Library)

Must contain:

- Models (All Entities)
- DbContext
- Repository Pattern (Interface + Implementation)
- EF Core Migrations

◊ Layer 2: API Service Layer (Web API)

Must contain:

- Controllers
- Authentication (JWT)
- Authorization
- Dependency Injection

⚙️ Technology Stack

- .NET 8
- ASP.NET Core Web API
- Entity Framework Core (Code First)
- SQL Server
- JWT Authentication

🔗 Mandatory Implementation Rules

- ✓ Use **Repository Pattern**
- ✓ Use **Dependency Injection**
- ✓ Use **Async/Await**
- ✓ Use **EF Core Migrations**
- ✓ Configure **Connection String**
- ✓ Implement **JWT Authentication**
- ✓ Implement **Role-Based Authorization**

Expectations

🔗 Implementation Expectations

Students must:

- ✓ Implement DbContext with DbSet

- ✓ Configure relationships
- ✓ Implement Repository Pattern
- ✓ Configure JWT in Program.cs
- ✓ Use:
 - [Authorize]
 - [Authorize(Roles = "Admin")]
- ✓ Follow REST API conventions:
 - GET /api/contacts
 - GET /api/contacts/{id}
 - POST /api/contacts
 - PUT /api/contacts/{id}
 - DELETE /api/contacts/{id}
- ✓ Return proper HTTP status codes:
 - 200 OK
 - 201 Created
 - 400 Bad Request
 - 404 Not Found

Learning Outcomes

After completing this task, students will:

- 🚀 Understand EF Core Code First Approach
- 🚀 Learn Database Migration & Schema Creation
- 🚀 Implement Repository Pattern
- 🚀 Understand Layered Architecture
- 🚀 Implement JWT Authentication
- 🚀 Implement Role-Based Authorization
- 🚀 Build secure and scalable APIs
- 🚀 Gain real-world backend development experience

Problem: Contact Management (Exception Handling & Logging)

Scenario:

The Contact Management System is now fully functional with CRUD operations, EF Core integration, and JWT-based authentication.

However, during testing and production usage, the application is facing issues such as:

- Unhandled exceptions causing application crashes
- Poor error responses sent to clients
- Difficulty in tracking errors and debugging issues

To improve reliability and maintainability, the organization wants to implement a **centralized Exception Handling and Logging mechanism**.

You are assigned to enhance the existing Web API by implementing **robust exception handling and structured logging**.

Requirements:

📌 Exception Handling

Students must implement **global exception handling** for the application.

☑ Functional Requirements

- Handle all unhandled exceptions centrally
- Return **standardized error response** to client
- Avoid exposing sensitive information (e.g., stack trace)

📌 Expected Error Response Format

```
{  
  "message": "Something went wrong",  
  "statusCode": 500,  
  "timestamp": "2026-04-05T10:00:00"  
}
```

📌 Types of Exceptions to Handle

- Custom exceptions (e.g., ContactNotFoundException)
- Validation errors
- Database-related exceptions
- General exceptions
-

📌 Logging

must implement logging for application activities and errors.

☑ Logging Requirements

- Log all exceptions
- Log important events:
 - Contact created
 - Contact updated
 - Contact deleted
 - User login attempts

Logging Details

Each log should include:

- Timestamp
- Log Level (Information, Warning, Error)
- Message
- Exception details (if any)

Logging Targets

- Console logging (mandatory)
- File logging (optional but recommended)

Technical Constraints

Mandatory Implementation

- Use **built-in logging framework (ILogger)**
- Implement **Global Exception Middleware OR Exception Filter**
- Create **Custom Exception Classes**
- Use **Dependency Injection**

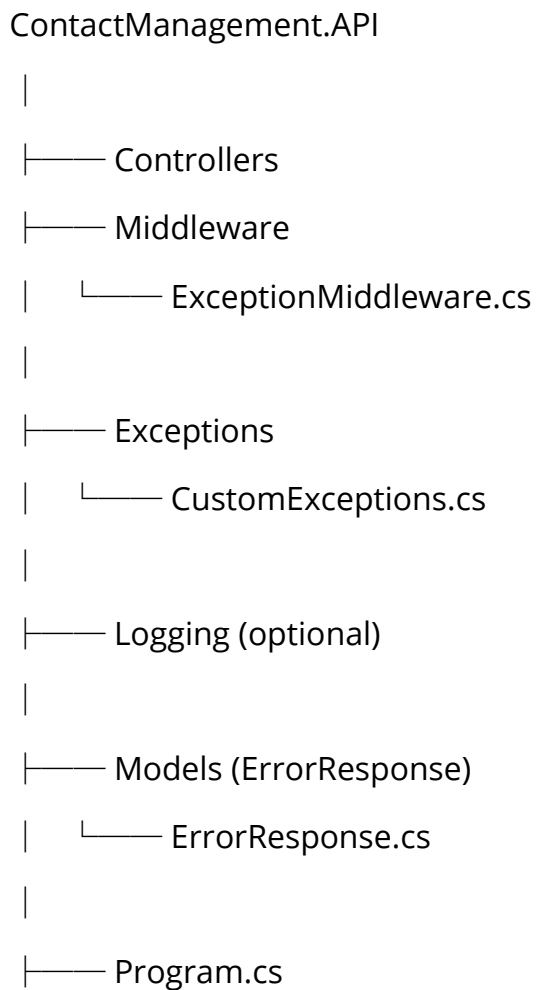
Restrictions

- Do NOT handle exceptions inside every controller method manually
- Do NOT expose stack trace in API response

- Do NOT ignore logging

Expectations

Suggested Structure



Implementation Expectations

Students must:

-  Create Global Exception Middleware
-  Create Custom Exception (e.g., NotFoundException)
-  Log errors using ILogger
-  Return consistent error response
-  Use appropriate HTTP status codes:
 - 400 → Bad Request
 - 404 → Not Found
 - 500 → Internal Server Error

Learning Outcomes

After completing this task, students will:

-  Understand Global Exception Handling
-  Learn Middleware in ASP.NET Core
-  Implement Structured Logging
-  Improve API reliability & debugging
-  Learn best practices for production-ready APIs
-  Implement Role-Based Authorization
-  Build secure and scalable APIs
-  Gain real-world backend development experience

CODE:

 ContactManagement.DAL/Models/Company.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ContactManagement.DAL.Models
{
    public class Company
    {
        public int CompanyId { get; set; }
        public string CompanyName { get; set; } = string.Empty;

        // Navigation property - one company has many contacts
        public ICollection<ContactInfo>? Contacts { get; set; }
    }
}
```

 ContactManagement.DAL/Models/Department.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ContactManagement.DAL.Models
{
    public class Department
    {
        public int DepartmentId { get; set; }
        public string DepartmentName { get; set; } = string.Empty;

        // Navigation property - one department has many contacts
        public ICollection<ContactInfo>? Contacts { get; set; }
    }
}
```

ContactManagement.DAL/Models/ContactInfo.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ContactManagement.DAL.Models
{
    public class ContactInfo
    {
        [Key]
        public int ContactId { get; set; }

        [Required]
        public string FirstName { get; set; } = string.Empty;

        [Required]
        public string LastName { get; set; } = string.Empty;

        [Required]
        [EmailAddress]
        public string EmailId { get; set; } = string.Empty;

        [Required]
        public long MobileNo { get; set; }

        public string Designation { get; set; } = string.Empty;

        // Foreign Keys
        public int CompanyId { get; set; }
        public int DepartmentId { get; set; }

        // Navigation properties
        [ForeignKey("CompanyId")]
        public virtual Company? Company { get; set; }

        [ForeignKey("DepartmentId")]
        public virtual Department? Department { get; set; }
    }
}
```

ContactManagement.DAL/Models/User.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ContactManagement.DAL.Models
{
    public class User
    {
        public int UserId { get; set; }
        public string Username { get; set; } = string.Empty;
        public string PasswordHash { get; set; } = string.Empty;
        public string Role { get; set; } = "User"; // Admin or User
    }
}
```

ContactManagement.DAL/DbContext/ApplicationDbContext.cs

```
using ContactManagement.DAL.Models;
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
```

```

using System.Text;
using System.Threading.Tasks;

namespace ContactManagement.DAL.DbContext
{
    public class ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : Microsoft.EntityFrameworkCore.DbContext(options)
    {
        public DbSet<ContactInfo> ContactInfos { get; set; }
        public DbSet<Company> Companies { get; set; }
        public DbSet<Department> Departments { get; set; }
        public DbSet<User> Users { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // Seed some initial data for Companies
            modelBuilder.Entity<Company>().HasData(
                new Company { CompanyId = 1, CompanyName = "Microsoft" },
                new Company { CompanyId = 2, CompanyName = "Google" },
                new Company { CompanyId = 3, CompanyName = "Amazon" }
            );

            // Seed Departments
            modelBuilder.Entity<Department>().HasData(
                new Department { DepartmentId = 1, DepartmentName = "IT" },
                new Department { DepartmentId = 2, DepartmentName = "HR" },
                new Department { DepartmentId = 3, DepartmentName = "Sales" }
            );

            // Seed a default Admin user (Password: admin123)
            // The hash is for "admin123" using simple Base64 encoding
            modelBuilder.Entity<User>().HasData(
                new User
                {
                    UserId = 1,
                    Username = "admin",
                    PasswordHash =
                        Convert.ToBase64String(System.Text.Encoding.UTF8.GetBytes("admin123")),
                    Role = "Admin"
                }
            );
        }
    }
}

```

ContactManagement.DAL/Repository/IRepository.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Linq.Expressions;
using System.Text;
using System.Threading.Tasks;

namespace ContactManagement.DAL.Repository
{
    public interface IRepository<T> where T : class
    {
        Task<T?> GetByIdAsync(int id);
        Task<IEnumerable<T>> GetAllAsync();
        Task<IEnumerable<T>> FindAsync(Expression<Func<T, bool>> predicate);
        Task<T> AddAsync(T entity);
        Task UpdateAsync(T entity);
        Task DeleteAsync(T entity);
        Task<bool> ExistsAsync(Expression<Func<T, bool>> predicate);
    }
}

```

📁 ContactManagement.DAL/Repository/Repository.cs

```
using ContactManagement.DAL.DbContext;
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Linq.Expressions;
using System.Text;
using System.Threading.Tasks;
namespace ContactManagement.DAL.Repository
{
    public class Repository<T>(ApplicationDbContext context) : IRepository<T> where T : class
    {
        protected readonly ApplicationDbContext _context = context;
        protected readonly DbSet<T> _dbSet = context.Set<T>();
        public async Task<T?> GetByIdAsync(int id)
        {
            return await _dbSet.FindAsync(id);
        }
        public async Task<IEnumerable<T>> GetAllAsync()
        {
            return await _dbSet.ToListAsync();
        }
        public async Task<IEnumerable<T>> FindAsync(Expression<Func<T, bool>> predicate)
        {
            return await _dbSet.Where(predicate).ToListAsync();
        }
        public async Task<T> AddAsync(T entity)
        {
            await _dbSet.AddAsync(entity);
            await _context.SaveChangesAsync();
            return entity;
        }
        public async Task UpdateAsync(T entity)
        {
            _dbSet.Update(entity);
            await _context.SaveChangesAsync();
        }
        public async Task DeleteAsync(T entity)
        {
            _dbSet.Remove(entity);
            await _context.SaveChangesAsync();
        }
        public async Task<bool> ExistsAsync(Expression<Func<T, bool>> predicate)
        {
            return await _dbSet.AnyAsync(predicate);
        }
    }
}
```

📁 ContactManagement.DAL/Repository/IContactRepository.cs

```
using ContactManagement.DAL.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ContactManagement.DAL.Repository
{
    public interface IContactRepository : IRepository<ContactInfo>
    {
        Task<IEnumerable<ContactInfo>> GetAllContactsWithDetailsAsync();
        Task<ContactInfo?> GetContactWithDetailsByIdAsync(int id);
    }
}
```

ContactManagement.DAL/Repository/ContactRepository.cs

```
using ContactManagement.DAL.DbContext;
using ContactManagement.DAL.Models;
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ContactManagement.DAL.Repository
{
    public class ContactRepository(ApplicationDbContext context) :
    Repository<ContactInfo>(context), IContactRepository
    {
        public async Task<IEnumerable<ContactInfo>> GetAllContactsWithDetailsAsync()
        {
            return await _context.ContactInfos
                .Include(c => c.Company)
                .Include(c => c.Department)
                .ToListAsync();
        }
        public async Task<ContactInfo?> GetContactWithDetailsByIdAsync(int id)
        {
            return await _context.ContactInfos
                .Include(c => c.Company)
                .Include(c => c.Department)
                .FirstOrDefaultAsync(c => c.ContactId == id);
        }
    }
}
```

ContactManagement.API/DTOs/ContactDto.cs

```
namespace ContactManagement.API.DTOs{

    public class ContactDto
    {
        public int ContactId { get; set; }
        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string EmailId { get; set; } = string.Empty;
        public long MobileNo { get; set; }
        public string Designation { get; set; } = string.Empty;
        public int CompanyId { get; set; }
        public string? CompanyName { get; set; }
        public int DepartmentId { get; set; }
        public string? DepartmentName { get; set; }
    }
    public class CreateContactDto {
        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string EmailId { get; set; } = string.Empty;
        public long MobileNo { get; set; }
        public string Designation { get; set; } = string.Empty;
        public int CompanyId { get; set; }
        public int DepartmentId { get; set; }
    }
    public class UpdateContactDto {
        public string FirstName { get; set; } = string.Empty;
        public string LastName { get; set; } = string.Empty;
        public string EmailId { get; set; } = string.Empty;
        public long MobileNo { get; set; }
        public string Designation { get; set; } = string.Empty;
        public int CompanyId { get; set; }
        public int DepartmentId { get; set; }
    }
}
```

📁 ContactManagement.API/DTOs/AuthDto.cs

```
namespace ContactManagement.API.DTOs
{
    public class RegisterDto
    {
        public string Username { get; set; } = string.Empty;
        public string Password { get; set; } = string.Empty;
        public string Role { get; set; } = "User";
    }

    public class LoginDto
    {
        public string Username { get; set; } = string.Empty;
        public string Password { get; set; } = string.Empty;
    }

    public class LoginResponseDto
    {
        public string Token { get; set; } = string.Empty;
        public string Username { get; set; } = string.Empty;
        public string Role { get; set; } = string.Empty;
    }
}
```

📁 ContactManagement.API/Exceptions/CustomExceptions.cs

```
namespace ContactManagement.API.Exceptions
{
    public class NotFoundException(string message) : Exception(message)
    {
    }

    public class BadRequestException(string message) : Exception(message)
    {
    }

    public class UnauthorizedException(string message) : Exception(message)
    {
    }
}
```

📁 ContactManagement.API/Models/ErrorResponse.cs

```
namespace ContactManagement.API.Models
{
    public class ErrorResponse
    {
        public string Message { get; set; } = string.Empty;
        public int StatusCode { get; set; }
        public DateTime Timestamp { get; set; }
    }
}
```

📁 ContactManagement.API/Middleware/ExceptionMiddleware.cs

```
using ContactManagement.API.Exceptions;
using ContactManagement.API.Models;
using System.Text.Json;

namespace ContactManagement.API.Middleware
{
    public class ExceptionMiddleware(RequestDelegate next, ILogger<ExceptionMiddleware> logger)
    {
        private readonly RequestDelegate _next = next;
        private readonly ILogger<ExceptionMiddleware> _logger = logger;

        public async Task InvokeAsync(HttpContext context)
        {
            try
```

```

    {
        await _next(context);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "An error occurred: {Message}", ex.Message);
        await HandleExceptionAsync(context, ex);
    }
}

private static async Task HandleExceptionAsync(HttpContext context, Exception exception)
{
    var response = new ErrorResponse
    {
        Timestamp = DateTime.UtcNow
    };

    switch (exception)
    {
        case NotFoundException:
            response.StatusCode = 404;
            response.Message = exception.Message;
            break;

        case BadRequestException:
            response.StatusCode = 400;
            response.Message = exception.Message;
            break;

        case UnauthorizedException:
            response.StatusCode = 401;
            response.Message = exception.Message;
            break;

        default:
            response.StatusCode = 500;
            response.Message = "Something went wrong. Please try again later.";
            break;
    }

    context.Response.StatusCode = response.StatusCode;
    context.Response.ContentType = "application/json";

    var jsonResponse = JsonSerializer.Serialize(response);
    await context.Response.WriteAsync(jsonResponse);
}
}
}

```

ContactManagement.API/Controllers/AuthController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using ContactManagement.DAL.DbContext;
using ContactManagement.DAL.Models;
using ContactManagement.API.DTOs;
using ContactManagement.API.Exceptions;

namespace ContactManagement.API.Controllers;

[ApiController]
[Route("api/[controller]")]
// Primary Constructor: Injection happens right here in the class header
public class AuthController(
    ApplicationDbContext context,
    IConfiguration configuration,

```

```

ILogger<AuthController> logger) : ControllerBase
{
    [HttpPost("register")]
    public async Task<IActionResult> Register(RegisterDto registerDto)
    {
        var userExists = await context.Users.AnyAsync(u => u.Username == registerDto.Username);
        if (userExists)
        {
            throw new BadRequestException("User already exists");
        }

        var user = new User
        {
            Username = registerDto.Username,
            PasswordHash = HashPassword(registerDto.Password), // Calling static method
            Role = registerDto.Role == "Admin" ? "Admin" : "User"
        };

        await context.Users.AddAsync(user);
        await context.SaveChangesAsync();

        logger.LogInformation("New user registered: {Username} with role {Role}", user.Username,
            user.Role);

        return Ok(new { message = "User registered successfully" });
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login(LoginDto loginDto)
    {
        var user = await context.Users.FirstOrDefaultAsync(u => u.Username ==
            loginDto.Username);

        if (user == null || !VerifyPassword(loginDto.Password, user.PasswordHash))
        {
            logger.LogWarning("Failed login attempt for user: {Username}", loginDto.Username);
            throw new UnauthorizedException("Invalid username or password");
        }

        var token = GenerateJwtToken(user);
        logger.LogInformation("User logged in: {Username}", user.Username);

        return Ok(new LoginResponseDto
        {
            Token = token,
            Username = user.Username,
            Role = user.Role
        });
    }

    private string GenerateJwtToken(User user)
    {
        // Note: This still uses 'configuration' from the primary constructor
        var securityKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(configuration["Jwt:Key"] ??
"ThisIsASecretKeyForJWTTokenThatIsAtLeast32CharsLong!"));
        var credentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256);

        var claims = new[]
        {
            new Claim(ClaimTypes.NameIdentifier, user.UserId.ToString()),
            new Claim(ClaimTypes.Name, user.Username),
            new Claim(ClaimTypes.Role, user.Role)
        };

        var token = new JwtSecurityToken(
            issuer: configuration["Jwt:Issuer"],
            audience: configuration["Jwt:Audience"],
            claims: claims,
            expires: DateTime.Now.AddDays(1),
            signingCredentials: credentials
    
```



```

    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}

// Marked as static because it only relies on the input string
private static string HashPassword(string password)
{
    return Convert.ToBase64String(Encoding.UTF8.GetBytes(password));
}

// Marked as static because it only relies on the input string and hash
private static bool VerifyPassword(string password, string hash)
{
    return Convert.ToBase64String(Encoding.UTF8.GetBytes(password)) == hash;
}
}

```

ContactManagement.API/Controllers/ContactsController.cs

```

using ContactManagement.API.DTOS;
using ContactManagement.API.Exceptions;
using ContactManagement.DAL.Models;
using ContactManagement.DAL.Repository;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace ContactManagement.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [Authorize]
    public class ContactsController(IContactRepository contactRepository,
        ILogger<ContactsController> logger) : ControllerBase
    {
        private readonly IContactRepository _contactRepository = contactRepository;
        private readonly ILogger<ContactsController> _logger = logger;

        [HttpGet]
        [Authorize] // Both Admin and User can view
        public async Task<IActionResult> GetAllContacts()
        {
            var contacts = await _contactRepository.GetAllContactsWithDetailsAsync();

            var contactDtos = contacts.Select(c => new ContactDto
            {
                ContactId = c.ContactId,
                FirstName = c.FirstName,
                LastName = c.LastName,
                EmailId = c.EmailId,
                MobileNo = c.MobileNo,
                Designation = c.Designation,
                CompanyId = c.CompanyId,
                CompanyName = c.Company?.CompanyName,
                DepartmentId = c.DepartmentId,
                DepartmentName = c.Department?.DepartmentName
            });

            return Ok(contactDtos);
        }

        [HttpGet("{id}")]
        [Authorize] // Both Admin and User can view
        public async Task<IActionResult> GetContactById(int id)
        {
            var contact = await _contactRepository.GetContactWithDetailsByIdAsync(id);

            if (contact == null)
            {
                throw new NotFoundException($"Contact with ID {id} not found");
            }
        }
    }
}

```

```

    }

    var contactDto = new ContactDto
    {
        ContactId = contact.ContactId,
        FirstName = contact.FirstName,
        LastName = contact.LastName,
        EmailId = contact.EmailId,
        MobileNo = contact.MobileNo,
        Designation = contact.Designation,
        CompanyId = contact.CompanyId,
        CompanyName = contact.Company?.CompanyName,
        DepartmentId = contact.DepartmentId,
        DepartmentName = contact.Department?.DepartmentName
    };

    return Ok(contactDto);
}

[HttpPost]
[Authorize(Roles = "Admin")] // Only Admin can create
public async Task<IActionResult> CreateContact(CreateContactDto createDto)
{
    var contact = new ContactInfo
    {
        FirstName = createDto.FirstName,
        LastName = createDto.LastName,
        EmailId = createDto.EmailId,
        MobileNo = createDto.MobileNo,
        Designation = createDto.Designation,
        CompanyId = createDto.CompanyId,
        DepartmentId = createDto.DepartmentId
    };

    var created = await _contactRepository.AddAsync(contact);

    _logger.LogInformation("Contact created: {FirstName} {LastName} by {User}",
        created.FirstName, created.LastName, User.Identity?.Name);

    return CreatedAtAction(nameof(GetContactById), new { id = created.ContactId },
created);
}

[HttpPut("{id}")]
[Authorize(Roles = "Admin")] // Only Admin can update
public async Task<IActionResult> UpdateContact(int id, UpdateContactDto updateDto)
{
    var existingContact = await _contactRepository.GetByIdAsync(id);

    if (existingContact == null)
    {
        throw new NotFoundException($"Contact with ID {id} not found");
    }

    existingContact.FirstName = updateDto.FirstName;
    existingContact.LastName = updateDto.LastName;
    existingContact.EmailId = updateDto.EmailId;
    existingContact.MobileNo = updateDto.MobileNo;
    existingContact.Designation = updateDto.Designation;
    existingContact.CompanyId = updateDto.CompanyId;
    existingContact.DepartmentId = updateDto.DepartmentId;

    await _contactRepository.UpdateAsync(existingContact);

    _logger.LogInformation("Contact updated: ID {ContactId} by {User}", id,
User.Identity?.Name);

    return Ok(new { message = "Contact updated successfully" });
}

[HttpDelete("{id}")]

```

```

[Authorize(Roles = "Admin")] // Only Admin can delete
public async Task<IActionResult> DeleteContact(int id)
{
    var contact = await _contactRepository.GetByIdAsync(id);

    if (contact == null)
    {
        throw new NotFoundException($"Contact with ID {id} not found");
    }

    await _contactRepository.DeleteAsync(contact);

    _logger.LogInformation("Contact deleted: ID {ContactId} by {User}", id,
        User.Identity?.Name);

    return Ok(new { message = "Contact deleted successfully" });
}
}
}

```

ContactManagement.API/Program.cs

```

using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
using ContactManagement.DAL.DbContext;
using ContactManagement.DAL.Repository;
using ContactManagement.API.Middleware;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

// Configure Database Connection
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// Configure Repository Pattern
builder.Services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
builder.Services.AddScoped<IContactRepository, ContactRepository>();

// Configure JWT Authentication
var jwtKey = builder.Configuration["Jwt:Key"] ??
    "ThisIsASecretKeyForJWTTokenThatIsAtLeast32CharsLong!";
var jwtIssuer = builder.Configuration["Jwt:Issuer"] ?? "ContactManagementAPI";
var jwtAudience = builder.Configuration["Jwt:Audience"] ?? "ContactManagementClient";

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwtIssuer,
            ValidAudience = jwtAudience,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey))
        };
    });

builder.Services.AddAuthorization();

// Add controllers
builder.Services.AddControllers();

// Add logging
builder.Services.AddLogging(logging =>
{

```

```

        logging.AddConsole();
        logging.AddDebug();
    });

    // Learn more about configuring Swagger/OpenAPI at https://aka.ms/aspnetcore/swashbuckle
    builder.Services.AddEndpointsApiExplorer();
    builder.Services.AddSwaggerGen(options =>
    {
        // Add JWT Authentication to Swagger
        options.AddSecurityDefinition("Bearer", new Microsoft.OpenApi.Models.OpenApiSecurityScheme
        {
            Name = "Authorization",
            Type = Microsoft.OpenApi.Models.SecuritySchemeType.Http,
            Scheme = "Bearer",
            BearerFormat = "JWT",
            In = Microsoft.OpenApi.Models.ParameterLocation.Header,
            Description = "Enter 'Bearer' followed by your token"
        });

        options.AddSecurityRequirement(new Microsoft.OpenApi.Models.OpenApiSecurityRequirement
        {
            {
                new Microsoft.OpenApi.Models.OpenApiSecurityScheme
                {
                    Reference = new Microsoft.OpenApi.Models.OpenApiReference
                    {
                        Type = Microsoft.OpenApi.Models.ReferenceType.SecurityScheme,
                        Id = "Bearer"
                    },
                    new string[] {}
                },
            }
        });
    });

    var app = builder.Build();

    // Configure the HTTP request pipeline.
    if (app.Environment.IsDevelopment())
    {
        app.UseSwagger();
        app.UseSwaggerUI();
    }

    // Add Global Exception Middleware
    app.UseMiddleware<ExceptionMiddleware>();

    app.UseHttpsRedirection();

    app.UseAuthentication(); // Add this before UseAuthorization
    app.UseAuthorization();

    app.MapControllers();

    // Ensure database is created and migrations are applied
    using (var scope = app.Services.CreateScope())
    {
        var dbContext = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
        dbContext.Database.EnsureCreated();
    }

    app.Run();

```

OUTPUT:

AUTH:

POST

/api/Auth/register

Parameters

No parameters

Request body

application/json

```
{
  "username": "newuser",
  "password": "newuser",
  "role": "Admin"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7285/api/Auth/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "newuser",
    "password": "newuser",
    "role": "Admin"
  }'
```

Request URL

https://localhost:7285/api/Auth/register

Server response

Code

Details

200

Response body

```
{
  "message": "User registered successfully"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 07 Apr 2025 12:14:27 GMT
server: Kestrel
```

Responses

Code	Description	Links
200	OK	No links

Reset

4

Clear

```
curl -X "POST" \
  'https://localhost:7295/api/Auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "newuser",
    "password": "newuser"
  }'
```

<https://localhost:7295/api/Auth/login>

Code	Details
------	---------

Response body

[illegible]

```
content-type: application/json; charset=utf-8
date: Tue, 07 Apr 2020 13:14:57 GMT
server: Restful
```

Links

No links

^

Cancel

Execute Clear

[illegible]

Code	Details
------	---------

```

{
  "contactId": 1,
  "firstName": "Vasu",
  "lastName": "Deva",
  "emailId": "vasu.deva@example.com",
  "mobileNo": 9876543210,
  "designation": "Software Engineer",
  "companyId": 1,
  "companyName": "Microsoft",
  "departmentId": 1,
  "departmentName": "IT"
}

```


[Download](#)

```
Response headers
content-type: application/json; charset=utf-8
date: Tue, 07 Apr 2020 12:17:30 GMT
server: Kestrel
```

Code	Description	Links
200	OK	No info

Reset

```
application/json
```


Clear

No links

Cancel

```
id = request.  
integer($int32) 1  
(path)
```

Clear

Cart

[illegible]

```
https://localhost:7295/api/Contacts/1
```

Code	Details
------	---------

```
{
  "contactId": 1,
  "firstName": "John",
  "lastName": "Doe",
  "emailId": "john.doe@example.com",
  "mobileNo": 9876543210,
  "designation": "Software Engineer",
  "companyId": 1,
  "companyName": "Microsoft",
  "departmentId": 1,
  "departmentName": "IT"
}
```

```
content-type: application/json; charset=utf-8
date: Tue, 04 Apr 2016 13:19:22 GMT
server: Mestrel
```

Links

No links

[illegible]

[illegible]

	ContactId	FirstName	LastName	EmailId	MobileNo	Designation	CompanyId	DepartmentId
	1	Vasu	Deva	vasu.deva@exa...	9876543210	Software Engin...	1	1
	2	Krishna	Devan	KrishnaDevan@...	9999955555	HR	2	1

	UserId	Username	PasswordHash	Role
	1	admin	AQAAAAIAAYa...	Admin
	2	keerthivasanrj	UGFzc3dvcnQx...	Admin
	3	newuser	bmV3dXNlcg==	Admin

	DepartmentId	DepartmentNa...
	1	IT
	2	HR
	3	Sales

	CompanyId	CompanyName
	1	Microsoft
	2	Google
	3	Amazon